Indian Institute of Technology, Gandhinagar

CS - 327 Compilers

Assignment 4

Intermediate Code Generation

Prof. Shouvick Mondal

Team:
Sailaja Mudu (23110288)
Harsha Vardhan Nayak (23110349)
Rushitha Thipparapu (23110338)

Contents

1. **Project Overview ……...……..……....3**
2. **Introduction ……..……....3**
3. **MiniC Language Specification……...…..4**
4. **Lexical Analyzer Design and Implementation …….…….....7**
5. **LALR(1) Parser Design and Implementation……..……..…15**
6. **Intermediate Code Generation (Three Address Code) ....……...28**
7. **Conclusion……..……...49**

Link:

- **https://github.com/HARSHA219219/Compiler-Designing.git**

1. Project Overview

This report documents the design, implementation, and testing of a front-end compiler for MiniC - a C-like language subset. The project spans four phases: lexical analysis, syntax analysis, syntax-directed translation, and intermediate code generation. The toolchain uses Flex for lexing and Bison/YACC for parsing, producing Three-Address Code (TAC) in quadruple form.

The pipeline is assembled as follows:

PHASE	TOOL/FILE	OUTPUT
Lexical Analysis	minic.l (Flex)	Token stream
Syntax Analysis	parser_resolved.y (Bison)	Parse tree / derivation
Semantic Actions	Embedded in grammar rules	Quadruple IR table
Error Handling	yyerror + error recovery	Diagnostics + partial TAC

2. Introduction

The design of a compiler begins with a precise and unambiguous language specification, because every later phase depends on the formal definition of valid source programs.
For this project, we designed MiniC, a simplified C-like imperative and procedural language specifically created for studying compiler construction concepts. Our language supports a carefully chosen subset of C constructs, including:

- primitive data types
- identifiers
- literals and constants
- arithmetic, relational, logical, and bitwise operators
- control-flow keywords
- delimiters and punctuators
- comments
- structured blocks

At the same time, we intentionally excluded several advanced C features, such as preprocessing directives, unions, compound assignments, shift operators, and advanced storage-class keywords, in order to keep the language suitable for progressive compiler development.
The purpose of this phase is to formally define the valid symbols, lexical categories, tokenisation rules, and scope boundaries of MiniC, so that the later phases of lexical analysis, syntax analysis, and syntax-directed translation can be built on a strong and consistent foundation.

3. MiniC Language Specification

3.1 Overview

MiniC is the C-like language subset designed for this compiler project. It preserves the familiar syntax of C while restricting its scope to make step-by-step compiler construction tractable. The result is a language that is expressive enough to represent meaningful programs yet simple enough to be fully analysed by an LALR(1) parser.

Core Design Principles :

- Imperative programming style — sequential statement execution
- Procedural execution model — functions as first-class organisational units
- ASCII-based source representation — printable and control characters only
- Case-sensitive token recognition — identifiers and keywords are distinct by case
- Whitespace-insensitive token separation — spaces, tabs, and newlines carry no semantic weight

3.2 Alphabet and Character Set

Every MiniC source program is defined as a sequence of characters drawn from a restricted ASCII-based alphabet (Σ). The four supported character categories are shown below.

Category	**Tokens/Examples**
Letters	a – z (lowercase) A – Z (uppercase)
Digits	0 -9
Special Symbols	+ - * / = < > ( ) { } ; , . _ @ # ! & | ^ ~ % [ ] :
Whitespace	space (0x20) tab (0x09) newline (0x0A) carriage-return (0x0D)

Whitespace rule: Whitespace is used only to delimit tokens and improve readability. It has no effect on program semantics and is discarded during lexical analysis. Any character outside the defined alphabet is flagged as a *LEXICAL_ERROR* token.

3.3 Token Categories

The MiniC lexer partitions source text into six distinct token categories. Each category is associated with a regular expression that the Flex-generated scanner uses to match input.

3.3.1 Keywords

int	float	char	void	long
short	double	if	else	while
do	for	switch	case	default
break	cntinue	return	struct	sizeof

3.3.2 Identifiers

We define identifiers using the regular expression:
[a-zA-Z_][a-zA-Z0-9_]*

This ensures that identifiers:

- Must begin with a letter *(a–z, A–Z)* or an underscore **(_).**
- May contain letters, digits, and underscores after the first character.
- Are case-sensitive: count and Count are distinct identifiers.
- Must not coincide with any reserved keyword.
- Have no enforced maximum length in the lexer.

3.3.3 Literals

MiniC supports five literal forms, covering the most common constant value representations in C-like programs.

Category	**Tokens/Examples**
Decimal Integer	42 0 1000 → INTEGER_LITERAL
Hexadecimal int	0x1A 0xFF 0X00 → INTEGER_LITERAL
Floating point	3.14 1.0e-3 2.5E+6 → FLOAT_LITERAL
Character const	'A' '\n' '\t' '\\' → CHAR_LITERAL
String literal	"hello" "world\n" → STRING_LITERAL

3.3.4 Operators

MiniC defines six operator families to support arithmetic computation, comparisons, logical reasoning, bit manipulation, and assignment. Operator precedence is encoded structurally in the grammar — not through % precedents directives — ensuring unambiguous parsing.

Family	**Operators**	**Notes**
Arithmetic	+ - * / %	Standard binary arithmetic; % is modulo
Relational	< > <= >= == !=	Return a boolean-valued temporary in TAC
Logical	&& || !	Short-circuit semantics (not enforced in TAC phase)
Bitwise	& | ^ ~	Bitwise AND, OR, XOR, NOT — lexed but not TAC-mapped yet
Assignment	=	Right-associative; generates '=' quadruple
Increment/Decrement	++ –	Prefix/postfix — lexed; TAC expansion in progress
Spaceship(custom)	⇔	MiniC extension — see Section 33.5 below

3.3.5 Custom Extension: Spaceship Operator

<=> Three-Way Comparison Operator

As a deliberate language design extension, MiniC introduces the <=> operator — the three-way comparator — which does not exist in standard C89/C90 (the C version that MiniC is modelled after). It returns a signed integer:

- −1 when left < right
- 0 when left == right
- +1 when left > right

This extension differentiates MiniC from standard C and demonstrates intentional language design beyond the minimal required feature set.

3.3.6 Delimiters and Comments

Delimiter	Symbol(s)	Purpose
Parentheses	( )	Groups expressions; encloses conditions and parameter lists
Braces	{ }	Delimits compound statements and function bodies
Brackets	[ ]	Array subscript access (lexed; not yet TAC-mapped)
Colon	:	Case labels in switch statements

3.3.7 Comments

- Single-line: // comment text — extends to end of line; discarded by lexer.
- Multi-line: /* comment text */ — may span multiple lines; uses an exclusive Flex start condition.
- Improperly closed block comments (file ends while still inside /* ... */) are reported as a LEXICAL ERROR via the <<EOF>> rule.
- Both comment forms are consumed and discarded during lexical analysis. They never reach the parser and generate no token.

3.4 Unsupported Features

MiniC is intentionally scoped to keep the compiler pipeline manageable for academic study. The following features present in full C have been deliberately excluded. Each exclusion is a conscious design choice that allows the project to focus on core compiler construction concepts :

Excluded Feature	Rationale
Preprocessor directives	#include, #define — require a separate pre-processing pass
Ternary operator	cond ? a : b — grammatically ambiguous without dedicated rules

Compound assignments	+=, -=, *= etc. — syntactic sugar; omitted for grammar simplicity
Shift Operators	<< >> — bitwise shifts; not needed for control-flow experiments
Unions & enums	union, enum — type-system complexity beyond project scope
Variadic functions	printf-style ... arguments — require runtime ABI support
Flexible arrays	struct s { int a[]; } — require dynamic memory semantics
Storage class keywords	static, extern, register, volatile — orthogonal concern

This controlled scope is not a limitation — it is a deliberate pedagogical choice. By narrowing the language surface, the compiler pipeline can be built incrementally across the semester: lexer → parser → semantic actions → TAC → (future) code generation. Each stage remains comprehensible in isolation, and the cumulative result is a complete, working front-end compiler for a non-trivial programming language.

4. Lexical Analyzer Design and Implementation

After formally defining the MiniC language specification, the next step was to implement the lexical analysis phase — the first executable stage of the compiler pipeline. In this phase, we developed a lexical analyzer using FLEX (Fast Lexical Analyzer Generator).

The lexical analyzer reads the MiniC source program as a stream of characters and groups them into meaningful lexical units called tokens. Each token corresponds to a logical program element, as shown in the table below.

Token category	**Description**
Keywords	Reserved words with fixed language meaning
Identifiers	User-defined names for variables and functions
Literals	Constant values: integers, floats, chars, strings
Operators	Arithmetic, relational, logical, bitwise, assignment
Punctuators	Structural delimiters: ; , ( ) { } [ ] :
Comments	Inline and block comments — discarded by lexer
Invalid Tokens	Unrecognised characters — flagged as lexical errors

The lexer implementation strictly follows the MiniC language specification defined in Section 3, ensuring consistency between the formal specification and the practical scanner design. The implementation is structured across three interconnected layers:

The screenshots below illustrate the core implementation flow of our FLEX-based lexical analyser.

```
DIGIT       [0-9]
LETTER      [a-zA-Z_]
ID          {LETTER}({LETTER}|{DIGIT})*
HEX         0[xX][0-9a-fA-F]+
INT         {DIGIT}+
FLOAT       {DIGIT}+\.{DIGIT}+([eE][+-]?{DIGIT}+)?
CHARCONST   \'(\\.|[^\\\'])\'
STRINGCONST \"([^\\\"\n]|\\.)*\"
```

In **this figure**, we define reusable macro patterns such as DIGIT, LETTER, ID, HEX, FLOAT, CHARCONST, and STRINGCONST. These abstractions improve maintainability and ensure that complex token definitions remain modular and readable.

```
{FLOAT} {
    print_token("FLOAT_LITERAL", yytext);
    float_count++;
}

{HEX} {
    print_token("INTEGER_LITERAL", yytext);
    int_count++;
}

[0-9]+[a-zA-Z_][a-zA-Z0-9_]* {
    print_token("INVALID_IDENTIFIER", yytext);
    invalid_identifier_count++;
    invalid_count++;
}

{INT} {
    print_token("INTEGER_LITERAL", yytext);
    int_count++;
}

{CHARCONST} {
    print_token("CHAR_LITERAL", yytext);
    char_count++;
}

{STRINGCONST} {
    print_token("STRING_LITERAL", yytext);
    string_count++;
}

{ID} {
    if (is_keyword(yytext)) {
        print_token("KEYWORD", yytext);
        keyword_count++;
    } else {
        print_token("IDENTIFIER", yytext);
        identifier_count++;
        add_symbol(yytext);
    }
}
```

In **this figure**, we implement the primary rule-action pairs for token recognition. Each regular-expression rule is directly associated with an action block that:

- prints token details
- updates token statistics
- validates malformed identifiers
- performs keyword verification
- inserts valid identifiers into the symbol table

This phase forms the **main token classification engine** of our lexical analyzer.

```
"<=>"|"<="|">="|"=="|"!="|"++"|"--"|"&&"|"||" {
    print_token("OPERATOR", yytext);
    operator_count++;
}

[+\-*/%=<>!&|^~.] {
    print_token("OPERATOR", yytext);
    operator_count++;
}

[(){}\[\];,:] {
    print_token("PUNCTUATOR", yytext);
    punctuator_count++;
}

[ \t\r]+ { column_number += strlen(yytext); }

. {
    print_token("INVALID_TOKEN", yytext);
    invalid_count++;
}
```

In **this figure,** we extend the lexer to handle:

- multi-character operators
- single-character operators
- punctuators
- whitespace skipping
- catch-all invalid token detection

The final `.` rule acts as a robust fallback error detector, ensuring that every unrecognised character is immediately reported as an `INVALID_TOKEN`.

Together, these three implementation layers demonstrate the complete scanner workflow from token pattern specification to lexical diagnostics, proving that the MiniC lexical analyzer is both specification-compliant and implementation-ready.

4.1 Token Recognition Strategy

To ensure robust tokenization, the lexical analyzer uses regular-expression based rules in FLEX. The scanner processes the source code character by character and uses pattern matching to classify each lexeme into the appropriate token category.

For every matched token, the analyzer systematically performs the following actions:

Action	**Mechanism**	**Purpose**
Classify token	Pattern matching	Assign the correct TOKEN type to the lexeme
Print token details	Formatted fprintf	Display type, lexeme, line, and column
Update counters	Per-category counters	Track keyword, identifier, literal counts
Symbol table insert	Hash-based lookup/insert	Store identifiers for use by later phases
Line/column tracking	mark_token() call	Capture precise source location metadata

This systematic design ensures that the lexical analyzer acts simultaneously as a scanner, a lexical statistics engine, and a diagnostic tool — providing rich context for all downstream compiler phases.

4.2 Token Categories Implemented

Based on the MiniC specification, the following token classes are implemented in the lexer. Each subsection describes the recognition strategy and key design decisions made during implementation.

4.2.1 Keywords

Instead of separate regex rules for each keyword, we used an efficient two-stage keyword recognition strategy:

Stage 1 — Match identifier pattern: *[a-zA-Z_][a-zA-Z0-9_]**

Stage 2 — Verify against keyword table: strcmp() lookup in a reserved-word array

This approach improves maintainability and scalability — adding a new keyword requires only a table entry, not a new regex rule. The 20 supported keywords are:

int	float	char	void	long
short	double	if	else	while
do	for	switch	case	default
break	cntinue	return	struct	sizeof

4.2.2 Identifiers

Identifiers are matched using:

[a-zA-Z_][a-zA-Z0-9_]*

This ensures:

- Must begin with a letter (a–z, A–Z) or an underscore (_)
- May contain letters, digits (0–9), and underscores after the first character
- Are case-sensitive — count and Count are distinct identifiers
- Must not coincide with any reserved keyword — checked in Stage 2
- Matched identifiers are inserted into the symbol table on first occurrence

4.2.3 Literals

The lexer supports five literal forms, covering all common constant value representations in C-like programs.

Literal Type	Pattern / Example
Decimal Integer	42 0 255
Hexadecimal int	0x1A 0xFF 0X00
Floating-point	3.14 1.0e-3 2.5E+6
Character const	'a' '\n' '\t' '\\'
String literal	"Hello" "world\n"

4.2.4 Operators and Delimiters

The lexer recognises a complete set of operator and delimiter tokens, covering all families defined in the MiniC specification

Family	Symbols	Token Names
Arithmetic	+ - * / %	PLUS MINUS MUL DIV MOD
Relational	< > <= >= == !=	LT GT LE GE EQ NE
Logical	&& || !	AND OR NOT
Assignment	=	ASSIGN
Increment/Decrement	++ --	INC_OP DEC_OP
Delimiters	) { } [ ] ; , :	LPAREN RPAREN … COLON
Extension	⇔	SPACESHIP OP

4.2.5 Rule Ordering Strategy

A critical part of the lexical analyzer design was the deliberate ordering of FLEX rules. FLEX resolves ambiguity using two principles:

- Longest match rule — the rule that matches the most characters wins.
- First rule priority — when two rules match the same length, the one listed first wins.

We ordered rules in the following sequence to prevent incorrect tokenization:

Priority	Rule Group	Rationale
1	Error-specific rules	Detect known malformed patterns before generics
2	Complex literals	Float & hex before plain integers
3	Multi-character operators	<=> <= >= == != && || before single chars
4	Invalid token patterns	Catch known bad patterns explicitly
5	Simple literals	Decimal integers, char & string constants
6	Identifiers / keywords	After literals to avoid shadowing
7	Single-character operators	+ - * / < > = ! …
8	Punctuators	( ) { } [ ] ; , :
9	Whitespace	Skip silently — advance line/column counters
10	Catch-all . rule	Last resort INVALID_TOKEN for any unmatched char

```
{FLOAT}
{HEX}
[0-9]+[a-zA-Z_][a-zA-Z0-9_]*
{INT}
{CHARCONST}
{STRINGCONST}
{ID}
"<=>"|"<="|">="|"=="|"!="|"++"|"--"|"&&"|"||"
[+\-*/%=<>!&|^~.]
[(){}\[\];,:]
.
```

Figure: Ordered FLEX rule priority used to ensure correct longest-match tokenization

This ordering was especially important for correctly distinguishing:

- == from = (assignment vs. equality)
- != from ! (not-equal vs. logical-not)
- <= and >= from < and > (compound relational)
- <=> from <= (spaceship vs. less-than-or-equal)
- && and || from & and | (logical vs. bitwise)

4.3 Symbol Table Generation

To strengthen the lexical analysis phase, symbol table support is integrated directly into the lexer. The symbol table records every unique identifier encountered during scanning, providing a foundation for later compiler phases .
Whenever a valid identifier is recognized:

Event	**Action**	**Data Stored**
New identifier matched	Insert into symbol table	Name, first-occurrence line number
Duplicate identifier	Skip insertion (no-op	-
Symbol table query	Lookup by name	Returns existing entry or NULL
Phase hand-off	Table passed to parser	Available for semantic analysis

A sample symbol table output generated by our lexer contains:

```
=========== SYMBOL TABLE ===========
Identifier           First Line
-------------------------------------
x                    1
_x1                  3
variable_name_123    11
another_var_456      11
y                    17
z                    17
_abc                 23
abc_123              23
variable_name_12345  33
anotherVeryLongIdentifier_999 33
z3                   37
integer              39
returns              39
_x                   49
```

Figure: Symbol table generated during integrated compiler execution showing unique identifiers and first occurrence lines

4.4 Error Handling in Lexical Analysis

The lexical analyzer is designed to provide strict and meaningful lexical diagnostics. Rather than crashing on unrecognised input, the lexer classifies and reports each error individually, allowing scanning to continue for the remainder of the file.

Error Category	**Detection Mechanism**
Invalid identifier (e.g. 1y, 12abc)	Digit-leading sequence matched before ID rule
Invalid character (e.g. @, #)	Catch-all . rule returns INVALID_TOKEN
Unterminated string literal	Newline inside string constant triggers error rule
Unclosed block comment	<<EOF>> rule inside COMMENT start condition
Malformed token (e.g. 0xGG)	Hex pattern fails; suffix caught as invalid

Error Output Format:

```
    ===== LEXICAL ERROR =====

Token   : INVALID_TOKEN
Lexeme  : '@'
Line    : 4

Column  : 12
```

This ensures strict compliance with the MiniC lexical specification — every character in the input is accounted for and either consumed as a valid token or reported as a precise, located error.

4.5. Lexical Analysis Output Summary

The lexical analysis phase generates a structured token stream summary that provides a clear view of how the MiniC source program is interpreted at the scanning stage.Our lexical analyzer generates a detailed structured output table containing:

- serial number
- token type
- lexeme
- row number
- column number
- lexeme length

```
===== LEXICAL ANALYSIS SUMMARY =====
Total Tokens              : 35
Keywords                  : 6
Identifiers               : 6
Integer Literals          : 5
Float Literals            : 2
String Literals           : 0
Character Literals        : 1
Operators                 : 6
Punctuators               : 6
Invalid Identifiers       : 0
Unterminated Strings      : 0
Total Invalid Tokens      : 3
```

Figure: Generalised lexical analysis summary showing token distribution statistics for a MiniC program

The lexical analysis phase transforms a raw character stream into a well-structured token stream. It simultaneously performs scanning, error detection, symbol table construction, and statistical reporting. This rich output provides all the information needed by the parser — and by later compiler phases — to correctly interpret and translate the MiniC source program.

Transition to Syntax Analysis

After completing lexical analysis, the next phase in the compiler pipeline is syntax analysis. While the lexical analyzer validates individual tokens, it does not verify whether the sequence of tokens forms a syntactically correct program.

Therefore, in Assignment 3, we design a parser based on context-free grammar rules to validate program structure and enforce language syntax. The parser consumes the token stream generated by the lexical analyzer and constructs derivations based on grammar productions.

This transition marks the shift from regular language processing to context-free language processing in the compiler pipeline.

5. LALR(1) Parser Design and Implementation

5.1 Overview

After completing the lexical analysis phase, the next stage in the MiniC compiler pipeline is syntax analysis, where the sequence of tokens generated by the lexer is validated against the grammar of the language.

In this phase, we designed and implemented an LALR(1) parser using Bison, which constructs a bottom-up parser capable of recognizing valid MiniC programs. The parser reads the token stream and determines whether it can be derived from the grammar defined for MiniC.

To study parser behavior in depth, we adopted a two-stage approach:

- First, we intentionally designed an ambiguous grammar to observe conflict generation.
- Then, we systematically resolved all conflicts to obtain a deterministic LALR(1) parser.

This approach allowed us to practically understand how grammar design directly affects parser construction and correctness.

5.2 Construction of LALR(1) Automaton

Bison automatically generates the LALR(1) automaton from the provided grammar. The automaton represents the parser as a finite state machine over LR item sets

Automaton Components	**Description**
LR Item Sets	Each state represents a set of LR(0)/LR(1) items capturing parsing progress
Shift Transitions	State changes when the parser consumes the next input symbol
Goto Transitions	State changes after a reduction — driven by the non-terminal produced
ACTION Table	Maps (state, terminal) → shift / reduce / accept
GOTO Table	Maps (state, non-terminal) → next state

For the original MiniC grammar:

- Total states generated: 79
- Parsing strategy: Bottom-up (Shift-Reduce parsing)

The parser construction clearly demonstrated how grammar ambiguity results in multiple possible parsing actions within the same state.

5.3 Conflict Identification

During parser generation for the original grammar (parser.y), Bison reported:

- 21 shift/reduce conflicts
- 0 reduce/reduce conflicts

All conflicts were analysed using the parser.output file produced by Bison. Three root causes were identified.

Conflict Source	Affected States	Conflict Type	Root Cause
Arithmetic expression ambiguity	States 39–42	Shift/Reduce	No operator precedence or associativity encoded
Assignment expression ambiguity	State 37	Shift/Reduce	Recursive production allows multiple parse trees
Dangling else ambiguity	State 71	Shift/Reduce	Two if-productions at same grammar level

5.3.1 Arithmetic Expression Ambiguity

The original grammar defined expressions as:

expression → expression + expression
expression → expression - expression
expression → expression * expression
expression → expression / expression

Since all operators were placed at the same level, the grammar did not encode operator precedence or associativity.

For example:

a + b * c

The parser could interpret this as:

- (a + b) * c
- a + (b * c)

The parser cannot decide whether to interpret this as (a + b) * c or a + (b * c). This ambiguity manifested as shift/reduce conflicts in States 39–42, where after recognising a partial expression, the parser could not choose between:

- Reducing the current expression (treating it as complete), or Shifting the next operator (extending the expression further

```
State 39

    35 expression: expression . PLUS expression
    35           | expression PLUS expression .
    36           | expression . MINUS expression
    37           | expression . MUL expression
    38           | expression . DIV expression

     PLUS   shift, and go to state 31
     MINUS  shift, and go to state 32
     MUL    shift, and go to state 33
     DIV    shift, and go to state 34

     PLUS      [reduce using rule 35 (expression)]
     MINUS     [reduce using rule 35 (expression)]
     MUL       [reduce using rule 35 (expression)]
     DIV       [reduce using rule 35 (expression)]
     $default  reduce using rule 35 (expression)
```

Figure 5.1: Shift/Reduce conflict in State 39 due to ambiguous arithmetic expression grammar.

In State 39, the parser encounters a shift/reduce conflict while parsing arithmetic expressions. After recognizing a partial expression such as "expression PLUS expression", the parser cannot decide whether to reduce the current expression or shift the next operator (PLUS, MINUS, MUL, DIV).

This conflict arises because all arithmetic operators are defined at the same grammar level, resulting in no precedence or associativity rules. As a result, expressions like "a + b * c" become ambiguous, leading to multiple possible parse trees.

5.3.2 Assignment Expression Ambiguity

The production:

expression → IDENTIFIER = expression

introduced ambiguity when followed by additional operators.

Example:

x = a + b

After parsing x = a, the parser could either:

- Reduce the assignment immediately, producing x = a as a complete expression, or
- Shift the + operator and extend the right-hand side.

This caused a shift/reduce conflict in State 37, because both actions were syntactically possible under the original grammar.

5.3.3 Dangling Else Ambiguity

The grammar rules:

if (expression) statement

if (expression) statement else statement

Introduced the classical dangling else problem.

Example:

```
if(a)
  if(b)
      x = 1;
  else
      x = 2;
```

The parser could not determine whether the else belongs to:

- the inner if, or
- the outer if

This resulted in a shift/reduce conflict in State 71.

```
State 71

  29 selection_statement: T_IF LPAREN expression RPAREN statement .
  30                    | T_IF LPAREN expression RPAREN statement . T_ELSE statement

   T_ELSE  shift, and go to state 73

   T_ELSE    [reduce using rule 29 (selection_statement)]
   $default  reduce using rule 29 (selection_statement)
```

Figure 5.2: Shift/Reduce conflict in State 71 demonstrating the dangling else problem.

In State 71, the parser encounters a shift/reduce conflict involving the "else" token. After parsing "if (expression) statement", the parser must decide whether to reduce the statement or shift the "else" token.

This ambiguity leads to the classic dangling else problem, where the parser cannot determine whether the "else" should be associated with the nearest unmatched "if" or an outer "if". This results in multiple possible interpretations of nested conditional statements.

Thus, all conflicts in the original grammar were shift/reduce conflicts, caused by:

- lack of operator precedence

- recursive ambiguity in assignment expressions
- dangling else ambiguity

These conflicts were intentionally preserved to study parser behavior before resolution.

Conflict Summary

All 21 conflicts in the original grammar were shift/reduce conflicts. They were caused by: (1) absence of operator precedence rules, (2) recursive ambiguity in assignment expressions, and (3) the dangling else problem. These conflicts were intentionally preserved at first to study parser behaviour before resolution.

5.4 Conflict Resolution

To eliminate all conflicts, the grammar was systematically refactored into a structured and precedence-aware form (parser_resolved.y).

5.4.1 Expression Grammar Refinement

We replaced the flat expression grammar with a **hierarchical structure**:

```
expression            → assignment_expression

assignment_expression → relational_expression
                      | IDENTIFIER = assignment_expression

relational_expression → additive_expression
                      | relational_expression relop additive_expression

additive_expression   → multiplicative_expression
                      | additive_expression + multiplicative_expression
                      | additive_expression - multiplicative_expression

multiplicative_expression → unary_expression
                          | multiplicative_expression * unary_expression
                          | multiplicative_expression / unary_expression

unary_expression      → primary_expression
                      | - unary_expression

primary_expression    → IDENTIFIER | INTEGER_LITERAL | FLOAT_LITERAL
                      | ( expression )
```

This restructuring:

- enforces operator precedence
- ensures correct associativity
- removes ambiguity in expression parsing

Property	Before(Parser.y)	After(Parser_resolved.y)
Operator precedence	Not encoded — all operators at same level	Encoded structurally via non-terminal hierarchy
Associativity	Undefined — leads to multiple parse trees	Left-associative through left-recursive productions
Assignment	Ambiguous recursive production	Right-associative via right-recursive production
Conflicts	21 shift/reduce conflicts	0 conflicts

5.4.2 Dangling Else Resolution

The dangling else conflict was resolved using the matched / unmatched statement split — the standard structural technique that avoids relying on %prec directives for nested if - else:

```
statement           → matched_statement
                    | unmatched_statement

matched_statement   → other_statement
                    | iteration_statement
                    | if_prefix matched_statement else matched_statement

unmatched_statement → if_prefix statement
                    | if_prefix matched_statement else unmatched_statement
```

The dangling else conflict was resolved using precedence directives in Bison:

%nonassoc LOWER_THAN_ELSE
%nonassoc T_ELSE

and assigning lower precedence to the if without else.

This ensures that:

- else is always associated with the nearest unmatched if
- the parser chooses shift over reduce in ambiguous cases

5.4.3 Result After Resolution

After applying these changes:

- Shift/Reduce conflicts: 0
- Reduce/Reduce conflicts: 0

This confirms that the grammar is LALR(1)-compatible and deterministic, while preserving the MiniC language semantics.

5.5 Parsing Table Generation

The parser generates two core tables:

Table	Inputs	Possible Entries	Role
ACTION	Current state + terminal token	Shift s_n / Reduce r_n / Accept	Drives the next parsing action on a terminal
GOTO	Current state + non-terminal	Next state number	Determines the next state after a reduction

ACTION Table

Determines parser actions:

- Shift
- Reduce
- Accept

State	int	float	char	void	id	=	(	)	,	int_lit	flt_lit	)	+	-	*	/	{	error	if	while	ret	}	INVALID_TOKEN	else	$
0	s1	s2	s3	s4																					acc
1																									r12
2																									r13
3																									r15
4																									r14
5																									acc
6	s1	s2	s3	s4																					r1
7																									r2
8																									r5
9																									r4
10					s13																				
11																									acc
12																									r3
13						s16	s17																		r10
14								s18	s19																
15																									r8
16					s20		s23			s21	s22														
17												s29													
18																									r7
19					s30																				
20						s32																			s42
21																									s44
22																									r68
23					s20		s23			s21	s22														
24																									s11
25													s34	s35											r34
26															s36	s37									r35
27																									r37
28																									r40
29																	s38								
30						s16																			r10
31																									r9
32					s20		s23			s21	s22														
33												s41													
34					s42		s23			s21	s22														
35					s42		s23			s21	s22														
36					s42		s23			s21	s22														
37					s42		s23			s21	s22														
38					s20		s23	s52		s21	s22						s38	s47	s48	s49	s50	s51	s53		
39																									r6
40																									r26
41																									r46
42																									r43
43															s36	s37									r38
44															s36	s37									r39
45																									r41
46																									r42
47								s52																	
48							s63																		
49							s64																		
50					s20		s23	s65		s21	s22														
51																									r17
52																									r28
53																									r25
54																									r20
55					s20		s23	s52		s21	s22						s38	s47	s48	s49	s50	s67	s53		
56																									r18
57																									r21
58																									r22
59																									r23
60																									r24
61								s69																	
62																									r26
63					s20		s23			s21	s22														
64					s20		s23			s21	s22														
65																									r33
66								s72																	
67																									r16
68																									r19
69																									r27
70												s73													
71												s74													
72																									r32
73					s20		s23	s52		s21	s22						s38	s47	s48	s49	s50		s53		
74					s20		s23	s52		s21	s22						s38	s47	s48	s49	s50		s53		
75																								s77	r29
76																									r31
77					s20		s23	s52		s21	s22						s38	s47	s48	s49	s50		s53		
78																									r30

Figure 5.3: ACTION table of the LALR(1) parser generated after grammar conflict resolution.

The ACTION table represents the parsing decisions taken by the LALR(1) parser for terminal symbols. Each entry specifies whether the parser performs a shift, reduce, or accept operation for a given state and input token.

Shift actions indicate that the parser consumes the next input symbol and transitions to a new state, while reduce actions correspond to the application of grammar productions. The accept action denotes successful completion of parsing.After grammar refinement, the ACTION table contains no conflicting entries, demonstrating that the parser operates deterministically without any shift/reduce or reduce/reduce conflicts.

GOTO Table

Defines transitions for non-terminals after reductions.

We implemented a custom parser table generator that processes the Bison .output file and displays:

- a structured ACTION table
- a structured GOTO table
- matrix format matching lecture standards

To improve readability, we introduced compact aliases such as:

- IDENTIFIER → id
- INTEGER_LITERAL → int_lit
- translation_unit → TU

This makes the parsing table more interpretable while preserving correctness.

State	Prog	TU	ExtDecl	FuncDef	Decl	Type	InitList	InitDecl	Expr	AssignExpr	AddExpr	MulExpr	PrimExpr	CompStmt	StmtList	Stmt	ExprStmt	SelStmt	IterStmt	JumpStmt
0	5	6	7	8	9	10														
1																				
2																				
3																				
4																				
5																				
6			12	8	9	10														
7																				
8																				
9																				
10							14	15												
11																				
12																				
13																				
14																				
15																				
16									24	25	26	27	28							
17																				
18																				
19								31												
20																				
21																				
22																				
23									33	25	26	27	28							
24																				
25																				
26																				
27																				
28																				
29														39						
30																				
31																				
32										40	26	27	28							
33																				
34												43	28							
35												44	28							
36													45							
37													46							
38									61	25	26	27	28	54	55	56	57	58	59	60
39																				
40																				
41																				
42																				
43																				
44																				
45																				
46																				
47																				
48																				
49																				
50									66	25	26	27	28							
51																				
52																				
53																				
54																				
55									61	25	26	27	28	54		68	57	58	59	60
56																				
57																				
58																				
59																				
60																				
61																				
62																				
63									70	25	26	27	28							
64									71	25	26	27	28							
65																				
66																				
67																				
68																				
69																				
70																				
71																				
72																				
73									61	25	26	27	28	54		75	57	58	59	60
74									61	25	26	27	28	54		76	57	58	59	60
75																				
76																				
77									61	25	26	27	28	54		78	57	58	59	60
78																				

Figure 5.5: GOTO table of the LALR(1) parser showing state transitions for non-terminals.

The GOTO table defines transitions between parser states after reductions are applied. Unlike the ACTION table, it operates on non-terminal symbols and determines the next state of the parser during the reduction phase.

Each entry maps a state and a non-terminal to a subsequent state, enabling the parser to continue the parsing process after applying a production rule.

Together with the ACTION table, the GOTO table forms the complete operational structure of the LALR(1) parser, ensuring correct and deterministic parsing behavior.

Key Properties of the Resolved Parsing Tables

- No cell in the ACTION table contains multiple entries — the parser is fully deterministic.
- Shift entries consume the current token and transition to a new state.
- Reduce entries apply a grammar production and pop states from the stack.
- The GOTO table is consulted after every reduction to restore the correct state for the non-terminal produced.

5.6 Reverse Derivation Tree

To visualise parser behaviour and validate correctness, reverse derivation tracking was implemented directly inside the grammar actions. Every reduction step records the production rule applied.

Phase	Action	Output
During parsing	Each reduce action records its production rule and a short description	Ordered list of reduction steps
After parsing	Derivation list is printed in reverse order	Rightmost derivation in reverse (bottom-up trace)
Validation use	Each P-rule maps to one grammar production	Proves the parser followed a valid parse sequence

During parsing:

- each reduction is recorded
- production rules are stored in sequence

After successful parsing:

- the derivation is printed in reverse order
- this corresponds to a rightmost derivation in reverse

This provides a clear trace of how the input program was reduced to the start symbol and helps validate parser correctness.

The successful execution of the parser and the generated reverse derivation tree are shown below:

```
Starting MiniC LALR(1) Parsing...


SUCCESS: Parsing completed successfully.

=== REVERSE DERIVATION TREE (Rightmost Derivation in Reverse) ===
Rule   | Production Content
-----------------------------------------------------------------
P1     | program -> translation_unit
P2     | translation_unit -> external_declaration
P5     | external_declaration -> function_definition
P6     | function_definition -> type IDENTIFIER () compound_statement
P16    | compound_statement -> { statement_list }
P18    | statement_list -> statement
P21    | statement -> expression_statement
P25    | expression_statement -> expression ;
P38    | primary_expression -> IDENTIFIER
P38    | primary_expression -> IDENTIFIER
P38    | primary_expression -> IDENTIFIER
P12    | type_specifier -> int
```

Figure 5.5: Successful parsing output showing reverse derivation sequence of the input program.

The above output demonstrates the successful parsing of the input program using the resolved LALR(1) grammar. After parsing completion, the parser prints the reverse derivation sequence, which corresponds to the rightmost derivation in reverse order.

Each production rule (P1, P2, etc.) represents a reduction step performed by the parser. This sequence directly reflects the reduce actions taken from the parsing table during execution.

The reverse derivation tree serves as a runtime trace of the parser and validates the correctness of the grammar and parsing process.

5.7 Error Diagnostics and Recovery

To enhance usability, the parser was equipped with advanced error diagnostics.

Features include:

- line number tracking
- column tracking
- offending token identification
- context-sensitive error messages

Example diagnostics:

- missing semicolon
- unmatched parentheses
- invalid assignment
- malformed expressions

We implemented panic-mode error recovery using semicolon-based synchronization, allowing:

- detection of multiple errors in one run
- continuation of parsing after errors
- partial derivation output

This significantly improves debugging capability while maintaining parsing efficiency.
The error diagnostics and recovery behavior of the parser is illustrated below:

```
=========== ERROR DIAGNOSTICS ===========
Message : syntax error
Line    : 2
Column  : 14
Token   : '<'
Hint    : Check statement syntax near the offending token.
=========================================

=========== ERROR DIAGNOSTICS ===========
Message : Syntax error: Recovering at next semicolon
Line    : 3
Column  : 18
Token   : ';'
Hint    : Missing expression or extra semicolon.
=========================================

=========== ERROR DIAGNOSTICS ===========
Message : syntax error
Line    : 5
Column  : 1
Token   : '}'
Hint    : Check statement syntax near the offending token.
=========================================
Starting MiniC LALR(1) Parsing...


FAILURE: Parsing failed due to syntax errors.
```

Figure 5.6: Error diagnostics output showing multiple syntax errors with location tracking and recovery.

The above output demonstrates the error diagnostics capability of the parser. When syntax errors are encountered, the parser reports detailed information including the error message, line number, column position, and the offending token.

Additionally, context-specific hints are provided to help identify the cause of the error, such as missing expressions or incorrect statement syntax. The parser also implements panic-mode error recovery by synchronizing at semicolon boundaries, allowing it to continue parsing after an error.

This enables the detection of multiple errors in a single run, improving debugging efficiency and usability of the parser.

5.8 Analytical Summary

The MiniC parser implementation demonstrates both theoretical and practical aspects of compiler design.

Initially, the ambiguous grammar produced multiple shift/reduce conflicts, clearly showing how grammar design directly impacts parser determinism.

Through systematic grammar restructuring:

- ambiguity was eliminated
- operator precedence was enforced
- dangling else was resolved
- deterministic parsing was achieved

This transformation highlights a fundamental compiler design principle:

Ambiguous grammars lead to parsing conflicts, and structured grammar refactoring produces deterministic LALR(1) parsers without changing the language semantics.

The final parser successfully integrates:

- conflict-free LALR(1) parsing
- parsing table visualization
- reverse derivation tracing
- robust error diagnostics

Thus, this phase completes the syntax analysis component of the MiniC compiler pipeline and provides a strong foundation for intermediate code generation in the next stage.

6. Intermediate Code Generation (Three Address Code)

After successfully constructing a deterministic LALR(1) parser in Assignment 3, the next stage of the MiniC compiler pipeline is Intermediate Code Generation, where syntactically valid programs are translated into a machine-independent representation.

In this phase, we extended the parser by integrating syntax-directed translation (SDT) into the grammar using Bison semantic actions. Unlike earlier phases that focused on recognition and validation, this phase performs actual program translation, converting MiniC source code into Three Address Code (TAC) represented as quadruples.

The primary objectives of this phase are:

- To transform high-level MiniC constructs into low-level intermediate instructions
- To explicitly represent control flow using jumps and labels
- To break complex expressions into simple operations
- To prepare the program for future optimization and target code generation

This phase bridges the gap between syntax analysis (Assignment 3) and machine-level code generation, making it a core component of compiler construction.

The correctness of this intermediate code generation phase is validated through multiple test cases, where the generated TAC accurately reflects the semantics of the input MiniC programs.

6.1 Design Approach and Parser Integration

The intermediate code generation phase in our MiniC compiler is tightly integrated with the LALR(1) parser implemented in parser_resolved.y. Instead of treating parsing and code generation as two separate stages, we adopted a syntax-directed translation (SDT) approach, where semantic actions are embedded directly within the grammar productions.

This design ensures that code generation occurs simultaneously with parsing, making the system both efficient and conceptually aligned with formal compiler design principles.

6.2.1 Core Design Strategy

The following key design decisions were adopted in our implementation:

- **Code generation during parsing**
  Intermediate code is generated *on-the-fly* as the parser processes the input. There is no separate traversal phase after parsing.
- **Semantic actions embedded in grammar rules**
  Each relevant grammar production contains C code that executes when the production is reduced. These actions are responsible for generating the corresponding Three Address Code (TAC).
- **Reduction-driven execution**
  Since Bison follows a bottom-up (shift-reduce) parsing strategy, semantic actions are executed during reduce operations. This means that code is generated only after a complete construct (such as an expression or statement) has been recognized.
- **Use of semantic value stack**
  The parser uses Bison's semantic value mechanism ($$, $1, $2, etc.) to:
  - pass intermediate results between grammar rules
  - store temporary variable names
  - propagate computed expressions upward in the parse tree

6.2.2 Working Mechanism

The integration works as follows:

1. The lexer provides a stream of tokens to the parser.
2. The LALR(1) parser processes tokens using ACTION and GOTO tables.
3. When a production rule is reduced, its associated semantic action is executed.
4. This semantic action:
   - generates one or more quadruples using the emit() function
   - creates temporary variables using new_temp() when required
   - assigns labels using new_label() for control flow constructs
5. The result of the computation is stored in $$ and passed upward for further use.

Thus, every reduction step in parsing corresponds directly to a code generation step, forming a seamless pipeline from syntax recognition to intermediate representation.

6.2.3 Example from Implementation

Consider the grammar rule for addition:

```
additive_expression:

  additive_expression PLUS multiplicative_expression

  {

    char *temp = new_temp();

    emit("+", $1, $3, temp);

    $$ = temp;

  }
```

Explanation:

- $1 represents the left operand
- $3 represents the right operand
- A new temporary variable is created
- A quadruple is emitted for the addition operation
- The result is stored in $$ and used in higher-level expressions

This demonstrates how semantic values and code generation are tightly coupled.

6.2.4 Advantages of This Approach

This integrated design provides several important benefits:

- **Eliminates need for Abstract Syntax Tree (AST)**
  Since code is generated during parsing, there is no requirement to construct and traverse an intermediate syntax tree.
- **Efficient execution**
  Parsing and code generation occur in a single pass, reducing time and memory overhead.
- **Direct mapping between grammar and semantics**
  Each production rule clearly defines both:
  - syntactic structure
  - semantic meaning
- **Simplified debugging and traceability**
  Since each reduction generates code, it is easier to trace how a particular TAC instruction was produced.

- **Consistency with compiler theory**
  This approach closely follows the concept of syntax-directed definitions (SDD) and translation schemes taught in compiler design.

6.2.5 Design Significance

From a theoretical perspective, this implementation demonstrates how:

- context-free grammar rules define program structure
- semantic actions attach meaning to those structures
- LALR(1) parsing can be extended beyond recognition to perform translation

Thus, the parser is no longer just a validator, but becomes an active translator, directly converting MiniC programs into an intermediate representation suitable for further compilation stages.

6.3 Quadruple-Based Intermediate Representation

In this phase of the MiniC compiler, we represent intermediate code using the Three Address Code (TAC) format, specifically implemented in the form of quadruples. This representation provides a structured and machine-independent way to express program logic, making it suitable for further optimization and target code generation.

6.3.1 Quadruple Structure

The quadruple is implemented in our system using the following C structure:

```
typedef struct {

  char op[16];

  char arg1[64];

  char arg2[64];

  char result[64];

} Quadruple;
```

Each quadruple corresponds to a single intermediate instruction, where all operations are reduced to at most three addresses (operands).

6.3.2 Storage Mechanism

All generated quadruples are stored sequentially in a global array:

Quadruple quadruples[2000];

int quad_count = 0;

- quadruples[] acts as the intermediate code buffer
- quad_count keeps track of the total number of generated instructions

Each time a new instruction is generated using the emit() function, it is appended to this array. This sequential storage preserves the execution order of the program, which is critical for maintaining correct semantics.

6.3.3 Components of a Quadruple

Each quadruple consists of four fields, each serving a specific purpose:

Field	Description
op	Specifies the operation to be performed (e.g., +, -, *, /, =, ifFalse, goto, label, return)
arg1	First operand (source value or variable)
arg2	Second operand (used for binary operations; may be empty for unary operations or assignments)
result	Destination where the result is stored

6.3.4 Types of Operations Represented

The quadruple representation in our implementation supports multiple categories of operations:

- **Arithmetic operations**
  +, -, *, /
- **Assignment operations**
  =
- **Unary operations**
  uminus (for negation)
- **Relational operations**
  <, >, <=, >=, ==, !=
- **Control flow operations**
  ifFalse (conditional jump)
  goto (unconditional jump)
  label (target location)

- **Function-related operations**
  return

This unified representation allows all program constructs—expressions, conditions, loops, and statements—to be expressed in a consistent format.

6.3.5 Example of Quadruple Generation

Consider the MiniC statement:

a = b + c;

This expression is decomposed into simpler operations during parsing, resulting in the following TAC:

t1 = b + c

a = t1

Corresponding quadruples:

op	arg1	arg2	result
+	b	c	t1
=	t1		a

6.3.6 Step-by-Step Translation

The translation process for the above example proceeds as follows:

1. The parser first recognizes the subexpression b + c
2. A temporary variable (t1) is created using new_temp()
3. The operation is emitted as:
   (+, b, c, t1)
4. The assignment is then processed:
   (=, t1, -, a)

This stepwise breakdown ensures that complex expressions are converted into simple, atomic operations, which is a key property of Three Address Code.

6.3.7 Integration with Emit Function

All quadruples are generated through the centralized function:

```
emit(op, arg1, arg2, result);
```

This function:

- constructs the quadruple
- safely copies operands into the structure
- appends it to the array
- ensures no buffer overflow occurs

This abstraction keeps code generation:

- modular
- reusable
- easy to maintain

6.3.8 Advantages of Quadruple Representation

The use of quadruples provides several important advantages:

- **Clarity and Structure**
  Each instruction is explicitly defined with fixed fields.
- **Ease of Translation**
  Can be directly mapped to assembly or machine code.
- **Optimization Friendly**
  Intermediate code can be easily analyzed and optimized.
- **Supports Complex Constructs**
  Control flow, expressions, and assignments are uniformly represented.
- **Debugging Support**
  The structured format makes it easy to trace execution and verify correctness.

6.3.9 Design Significance

From a compiler design perspective, the quadruple representation plays a critical role in bridging:

High-Level Source Code → Intermediate Representation → Target Code

It abstracts away machine-specific details while preserving program semantics, making it an ideal intermediate form.

6.4 Core Code Generation Mechanism

The generation of Three Address Code (TAC) in our MiniC compiler is built upon a set of **core utility functions** that systematically construct and manage intermediate instructions. These components ensure that code generation is consistent, modular, and tightly integrated with the parsing process.

The three primary elements of this mechanism are:

- the **emit function** for instruction generation
- **temporary variable management** for intermediate results
- **label generation** for control flow handling

Together, these components form the backbone of the intermediate code generation phase.

6.4.1 Emit Function

All intermediate instructions (quadruples) are generated using a centralized function:

```
static void emit(const char *op, const char *arg1, const char *arg2, const char *result)
```

Function Responsibilities

The emit() function performs the following operations:

- **Creates a new quadruple instruction**
  It constructs a TAC instruction using the provided operation and operands.
- **Stores the instruction in the global array**
  The generated quadruple is inserted into the quadruples[] array, preserving execution order.
- **Manages instruction indexing**
  The global counter quad_count is incremented to track the total number of instructions.
- **Ensures safe memory handling**
  String copying is done carefully using strncpy() to prevent buffer overflow.
- **Supports dual-mode generation (capture mode)**
  The function can store instructions either:
  - directly into the main quadruple list, or
  - temporarily into a **capture buffer** (used in for-loop handling)

Capture Mode Mechanism (Advanced Feature)

A notable feature of your implementation is the **capture mode**, which allows temporary storage of instructions:

```
if (capture_mode) {

  target = &captured_quads[captured_quad_count];

} else {

  target = &quadruples[quad_count];

}
```

This mechanism is particularly important for **for-loop translation**, where:

- increment expressions must be executed **after the loop body**
- but are parsed **before the loop body**

To handle this correctly:

- instructions are first captured
- then later flushed using flush_captured_quads()

This reflects a **correct and advanced implementation of loop semantics**, going beyond basic TAC generation.

Example

```
emit("+", "a", "b", "t1");
```

Generates the quadruple:

```
t1 = a + b
```

Design Significance

The emit() function centralizes all code generation logic, ensuring:

- uniform instruction formatting
- easier debugging
- maintainability and extensibility

6.4.2 Temporary Variable Generation

To handle intermediate computations, the compiler dynamically generates **temporary variables**, represented as:

```
t1, t2, t3, ...
```

These are created using the function:

```
static char *new_temp()
```

Working Mechanism

- A global counter (temp_count) is incremented
- A new name (t1, t2, etc.) is generated using sprintf()
- Memory is allocated using dup_text()

- The temporary variable is returned for use in expressions

Purpose of Temporary Variables

Temporary variables play a crucial role in TAC generation:

- **Store intermediate results**
  Complex expressions are broken into smaller steps, each stored in a temporary.
- **Maintain evaluation order**
  Ensures correct precedence and associativity during expression evaluation.
- **Simplify nested expressions**
  Deeply nested expressions are flattened into linear instructions.

Example

For:

```
a = (b + c) * d;
```

Generated TAC:

```
t1 = b + c

t2 = t1 * d

a = t2
```

Here:

- t1 stores (b + c)
- t2 stores the multiplication result

Design Advantage

Temporary variables ensure that:

- each TAC instruction has at most three operands
- complex expressions are decomposed into simple operations
- the representation remains consistent and easy to analyze

6.4.3 Label Generation

Control flow constructs such as **if statements, loops, and branching** require explicit jump targets. These are implemented using **labels**, represented as:

L1, L2, L3, ...

Labels are generated using:

```
static char *new_label()
```

Working Mechanism

- A global counter (label_count) is incremented
- A new label name (L1, L2, etc.) is created
- Memory is allocated and returned

Purpose of Labels

Labels are essential for representing control flow:

- **Define jump targets**
  Used by goto and ifFalse instructions.
- **Implement conditional branching**
  Direct execution based on boolean conditions.
- **Structure loops**
  Represent loop entry, exit, and iteration points.

Example (If Statement)

```
if (a < b)

  x = a;
```

Generated TAC:

```
t1 = a < b

ifFalse t1 goto L1

x = a

label L1
```

Example (Loop Structure)

```
while (a < b)
```

Generated TAC:

```
label L1
```

```
t1 = a < b

ifFalse t1 goto L2

body

goto L1

label L2
```

Design Significance

Labels transform structured control flow into:

- linear instruction sequences
- explicit jumps

This is essential for:

- machine-level execution
- control flow analysis
- optimization

6.4.4 Integrated Functionality

Together, the three components:

- emit()
- new_temp()
- new_label()

work in coordination to ensure that:

- every expression produces valid TAC
- every control structure is correctly represented
- intermediate results are properly stored
- execution flow is explicitly defined

6.5 Expression Handling

The intermediate code generator handles expressions using a **precedence-aware grammar structure**, ensuring that evaluation order strictly follows MiniC language rules. The implementation is based on layered non-terminals such as:

- multiplicative_expression
- additive_expression
- relational_expression
- assignment_expression

Each level corresponds to a specific operator precedence, enabling correct and unambiguous translation into Three Address Code (TAC).

6.5.1 Arithmetic Expressions

Arithmetic expressions are evaluated using a **hierarchical grammar**, where:

- Multiplicative operators (`*`, `/`) have higher precedence
- Additive operators (`+`, `-`) are evaluated afterward

This structure ensures that expressions are evaluated in a **bottom-up manner**, consistent with LALR(1) parsing.

Example (test1.c)

a = (b + c) * d;

Generated TAC

t1 = b + c

t2 = t1 * d

a = t2

Analysis

This output demonstrates:

- Correct enforcement of operator precedence
- Proper decomposition of expressions into smaller steps
- Use of temporary variables for intermediate results
- Left-to-right evaluation consistent with grammar rules

6.5.2 Unary Expressions

Unary operations, particularly negation, are handled using a dedicated rule in the grammar and translated using a special TAC operation:

emit("uminus", operand, "", result);

Example (test2.c)

a = -b + 5;

Generated TAC

t1 = minus b

t2 = t1 + 5

a = t2

Analysis

- Unary minus is evaluated before addition
- A temporary variable stores the negated value
- The result is then used in subsequent arithmetic operations

This ensures correct handling of unary operators within larger expressions.

6.5.3 Relational Expressions

Relational expressions generate **boolean-valued temporary variables**, which are later used in control flow instructions.

Example

t1 = a < b

Usage

These temporaries are essential for:

- Conditional statements (if, if-else)
- Loop conditions (while, for)

Analysis

- Each relational operation produces a temporary result
- The result is used by conditional jump instructions (ifFalse)
- This separates expression evaluation from control flow handling

6.6 Assignment Handling

Assignments are implemented using the grammar rule:

IDENTIFIER ASSIGN expression

Semantic Action

emit("=", $3, "", $1);

Where:

- $3 represents the evaluated expression
- $1 represents the destination variable

Example

x = a + b;

Generated TAC

t1 = a + b

x = t1

Analysis

- Expressions are evaluated first
- The final result is assigned to the target variable
- Ensures separation between computation and storage

This design simplifies code generation and aligns with standard TAC representation.

6.7 Control Flow Implementation

Control flow constructs are translated into TAC using:

- conditional jumps (ifFalse)
- unconditional jumps (goto)
- labels (L1, L2, …)

These constructs convert structured programming into **explicit control flow graphs**.

6.7.1 If Statement

An if statement is implemented using a single conditional jump.

Example

```
if (a < b)

  x = a;
```

Generated TAC

```
t1 = a < b

ifFalse t1 goto L1

x = a

label L1
```

Analysis

- Condition is evaluated first
- If false, control jumps to label L1
- Otherwise, the body is executed

6.7.2 If-Else Statement

The if-else construct requires:

- one conditional jump
- one unconditional jump
- two labels

Example (test20.c)

```
if (a < b)

  x = a;

else

  x = b;
```

Generated TAC

t1 = a < b

ifFalse t1 goto L1

x = a

goto L2

label L1

x = b

label L2

Analysis

- ifFalse directs control to the else block
- goto skips the else block after executing the if block
- Labels clearly separate control paths

6.7.3 While Loop

The while loop is implemented using a loop structure with entry and exit labels.

Example (test18.c)

while (a < b)

Generated TAC

label L1

t1 = a < b

ifFalse t1 goto L2

body

goto L1

label L2

Analysis

- L1 marks the loop start
- Condition is checked before each iteration

- goto L1 ensures repetition
- L2 marks loop termination

6.7.4 For Loop (Advanced Implementation)

The for loop implementation includes an advanced mechanism to correctly handle the execution order of its components.

Structure

for (init; condition; increment)

Key Mechanism: Quadruple Capture

The implementation introduces:

start_quad_capture();

stop_quad_capture();

flush_captured_quads();

Purpose

- Ensures that the **increment expression executes after the loop body**
- Maintains correct execution semantics
- Separates parsing order from execution order

Example (test10.c)

for (i = 0; i < limit; i = i + 1)

TAC Structure

- Initialization (i = 0)
- Loop condition check
- Body execution
- Deferred increment execution
- Loop back jump

Analysis

This implementation:

- Preserves correct loop semantics
- Demonstrates advanced control flow handling
- Reflects real compiler design techniques

The generated TAC for control structures has been verified using test cases, confirming correct execution flow and label-based branching behavior.

6.8 Nested Control Flow Handling

The parser supports complex nesting of control structures, including:

- nested if-else statements
- nested loops
- combinations of both

Example (test24.c)

Programs include:

- multiple loops
- deeply nested conditions
- mixed control constructs

Analysis

The generated TAC demonstrates:

- correct label scoping
- proper nesting of control structures
- accurate jump sequencing
- no ambiguity in execution flow

This confirms that the implementation correctly handles real-world program complexity.

6.9 Error Handling and Recovery

The intermediate code generator is integrated with a robust error-handling system inherited from the parser.

Features

- line number tracking
- column tracking
- offending token identification
- context-sensitive error messages
- panic-mode recovery

Example: Invalid Expression

x = a + * 2;

Output Behavior

- Parsing continues despite error
- Invalid expression is skipped
- Remaining valid code is processed

Example: Invalid Token

x = 10 @ 2;

Behavior

- @ is detected as an invalid token
- Parser recovers using synchronization
- Partial TAC is still generated

Analysis

- Ensures robustness of the compiler
- Allows multiple errors to be detected in a single run
- Prevents complete failure due to minor syntax issues

6.10 Output Representation

The intermediate code is displayed in a structured and readable tabular format.

Output Components

- Input source program
- Parsing status
- Generated TAC table

Format

| No | Statement | Op | Arg1 | Arg2 | Result |

Advantages

- Improves readability
- Simplifies debugging
- Clearly represents execution sequence
- Matches academic presentation standards

6.11 Reverse Derivation Integration

The parser continues to support **reverse derivation tracking** even after integrating code generation.

Functionality

- Each reduction step is recorded
- Production rules are stored
- Reverse derivation is printed after parsing

Significance

- Connects syntax analysis with code generation
- Provides traceability of parsing decisions
- Helps validate correctness of grammar and translation

6.12 End-to-End Compilation Example

To demonstrate the complete compiler pipeline, consider the following MiniC statement:

Input:
a = (b + c) * d;

Lexical Analysis:
IDENTIFIER ASSIGN (IDENTIFIER PLUS IDENTIFIER) MUL IDENTIFIER ;

Parsing:
The parser successfully recognizes the expression using precedence-aware grammar rules.

Generated TAC:
t1 = b + c

```
t2 = t1 * d
a = t2
```

This example illustrates how the compiler transforms high-level source code into structured intermediate representation through integrated lexical analysis, parsing, and code generation.

7. Conclusion

This project successfully demonstrates the complete design and implementation of a MiniC compiler front-end, developed incrementally across multiple phases of the compiler pipeline. Starting from formal language specification and progressing through lexical analysis, syntax analysis, and intermediate code generation, each stage was carefully designed to reflect both theoretical concepts and practical implementation techniques in compiler construction.

In the initial phase, a well-defined MiniC language specification was established, providing a clear foundation for all subsequent stages. By restricting advanced C features while preserving core language constructs, the design ensured both academic clarity and implementation feasibility.

The lexical analysis phase translated this formal specification into a working scanner using FLEX. The lexer efficiently tokenized input programs, handled complex patterns such as literals and multi-character operators, maintained a symbol table, and provided robust error detection. This phase ensured that only valid token streams were passed to the parser.

In the syntax analysis phase, an LALR(1) parser was implemented using Bison. A key highlight of this stage was the deliberate use of an ambiguous grammar to study shift/reduce conflicts, followed by systematic grammar refactoring to eliminate them. This provided practical insight into how ambiguity affects parser determinism and how structured grammar design leads to conflict-free parsing. The addition of reverse derivation tracking further strengthened the understanding of parser behavior and reduction sequences.

The final phase extended the parser into a syntax-directed translation system capable of generating intermediate code in the form of Three Address Code (TAC) quadruples. Code generation was tightly integrated with parsing, allowing semantic actions to execute during reduce operations. The implementation supports arithmetic expressions, relational operations, assignments, and complex control flow constructs including nested conditionals and loops.

A particularly significant achievement of this phase is the correct handling of control flow using labels and jumps, as well as the advanced implementation of the for loop using a quadruple capture mechanism. This ensures proper execution order and reflects real compiler design strategies. Additionally, robust error diagnostics and panic-mode recovery allow the system to continue processing even in the presence of syntax errors, improving usability and debugging capability.

Overall, the project effectively bridges the gap between theoretical compiler design and practical implementation. It demonstrates key principles such as:

- formal language specification
- regular expression-based tokenization
- context-free grammar design
- LALR(1) parsing and conflict resolution
- syntax-directed translation
- intermediate code representation
- error handling and recovery mechanisms

The resulting MiniC compiler front-end is not only functionally complete for its scope but also extensible. It provides a strong foundation for future enhancements such as semantic analysis, optimization techniques, and target code generation. Thus, the implemented system is not merely a theoretical model, but a fully functional MiniC compiler front-end capable of transforming source programs into structured intermediate representation with correct semantics and control flow.